

Ledger Sync — Solution & Deployment Notes

Simplify Money Backend Engineering Assignment

Prepared by Chandrama

This is a draft template built from the design decisions made so far — not a record of a confirmed working deployment. The commands below use placeholder values (module names, ports, image tags) that must be replaced with what your actual repo and build config use once the code is finished and run. Do not submit this without updating it to match a real, tested run.

1. Solution Summary

Ledger Sync ingests raw bank SMS/email, normalizes them into a deduplicated transaction ledger, categorizes every transaction into SPEND / INCOME / MICRO / TRANSFER, and writes three output files: ledger.json, summary.json, and reconciliation.json. A separate incident fix addresses a parser bug that misreads a whole-rupee transaction amount as the account balance. Storage is migrated from SQL to a document store, with a backfill job and a consistency checker to prove both stores agree.

Key design decisions

- Transaction identity = account_last4 + occurred_at (normalized to UTC) + direction + amount — not message_id — so re-ingesting overlapping or duplicate messages is a no-op by construction.
- Category accounting keeps MICRO and TRANSFER out of the spend/income totals in summary.json, while still listing MICRO transactions individually in ledger.json.
- Document store schema is built around the three declared query patterns (monthly transactions by account, running category totals, message-id lookup) rather than a ported relational shape.
- Anything the ledger can't account for is written to reconciliation.json with a specific reason, never silently dropped or forced to balance.

2. Architecture at a Glance

Pipeline stages:

- **Ingest** — read corpus-a.jsonl line by line.
- **Parse** — per-sender/per-template parsers (HDFC SMS, HDFC email, ICICI SMS, ICICI email) normalize each raw message into a candidate transaction fragment.
- **Accumulate** — fragments are merged by transaction identity key into a single NormalizedTxn, combining evidence (source_message_ids) from every message that witnesses the same event.
- **Categorize** — SPEND / INCOME / MICRO / TRANSFER assigned per the corpus rules.
- **Report** — ledger.json, summary.json, reconciliation.json written by the report command.
- **Store migration** — Backfill moves existing SQL rows into the document store; ConsistencyChecker diffs both stores field-by-field.

Placeholder module layout (replace with your actual package structure):

```
src/main/java/in/simplifymoney/ledgersync/  
  parse/    - HdfcSmsParser, IciciSmsParser, EmailParser, Amounts  
  model/    - NormalizedTxn (frozen), Category (frozen)  
  engine/   - TransactionAccumulator, CategoryClassifier, TransferDetector  
  report/   - Reports (ledger/summary/reconciliation writers)  
  store/    - Backfill, ConsistencyChecker, DocumentStore, SqlStore
```

3. How to Execute

Prerequisites (adjust versions to match your build.gradle / pom.xml):

- JDK — version per your build config (e.g. 17 or 21)
- Gradle or Maven — whichever the seed repo uses
- Docker + Docker Compose — for the document store and any supporting services

3.1 Clone and build

```
git clone <your-fork-url> ledger-sync  
cd ledger-sync  
./gradlew build    # or: mvn clean install
```

3.2 Run the test suite

```
./gradlew test    # or: mvn test  
# Confirm the INC-2026-09-11 regression test is present and passing.
```

3.3 Run the pipeline against corpus-a

```
./gradlew run --args="ingest fixtures/corpus-a.jsonl"  
./gradlew run --args="report output/"  
# Produces output/ledger.json, output/summary.json, output/reconciliation.json  
# Replace the above with your actual CLI entry point and argument names.
```

3.4 Verify against the checkpoint

```
diff <(jq -S . output/summary.json) <(jq -S . fixtures/corpus-a-totals.json)  
# Or use whatever comparison approach fits – the goal is confirming category totals reconcile  
to the paisa, per account.
```

4. Deployment Notes

The service and its document store must come up with a single documented command. Below is a placeholder docker-compose shape — replace image names, ports, and env vars with your real configuration once the document store choice (DynamoDB local or MongoDB) is finalized.

4.1 Example docker-compose.yml (placeholder — verify against your actual file)

```
services:  
  ledger-sync:  
    build: .
```

```
depends_on: [document-store]
environment:
  - DOCUMENT_STORE_URL=<set per your store>
ports: ["8080:8080"]
document-store:
  image: <mongo:7 OR amazon/dynamodb-local>
  ports: ["<store-port>:<store-port>"]
```

4.2 Bring the stack up

```
docker compose up --build
./verify.sh    # confirms the stack is healthy end to end
```

4.3 Document store choice

State your actual choice here (DynamoDB or MongoDB) with the one-paragraph justification tied to the three declared query patterns — see Section 3.4 of the project report for the reasoning already worked out. Include the six examined-vs-returned numbers (ScannedCount/Count or totalDocsExamined/nReturned) from a 100,000-transaction benchmark once that's been run.

4.4 Backfill and consistency check

```
# Run after the document store is up and the SQL source is populated:
./gradlew run --args="backfill"
./gradlew run --args="consistency-check"
# Backfill must be safe to re-run, including after a partial failure.
# ConsistencyChecker must report field-level, not row-count-level, discrepancies.
```

5. Known Gaps at Time of Writing

- ICICI dense-format SMS parsing, MICRO/TRANSFER accumulator fixes, Backfill, and ConsistencyChecker implementations are open — see the separate issue log for details.
- No end-to-end run has yet been confirmed against corpus-a.jsonl with output diffed against corpus-a-totals.json.
- Document store benchmarking (the six examined-vs-returned numbers) has not yet been run.
- docker compose up and ./verify.sh have not yet been confirmed working with the real service.

Update every placeholder (commands, ports, image names, module paths) to match your actual repository before this is submitted. A README section that describes a command that doesn't actually run is worse than no README section.